

Expresiones Regulares

El problema que resuelve un regex

Situación real

Tienen un dataset con una columna "teléfono" que tiene valores como:

```
099 123 456  
+598 99 123 456  
099-123-456  
(099) 123456  
tel: 099123456
```



Pregunta: "¿Cómo verifican que todos sean teléfonos válidos de Uruguay? ¿Cómo los normalizan a un solo formato?"

Con `if` y `.replace()` pueden... pero se vuelve un nudo imposible de desatar.

Con regex: una sola línea resuelve el problema.

¿Qué es una expresión regular?

Definición

Una expresión regular (regex) es un patrón de texto que describe un conjunto de cadenas posibles.

Analogía

Es como un molde de galletas para texto. Todo lo que calza con el molde es un *match*.

En Python se usa el módulo `re`.

Ejemplo en Python

```
import re

texto = "Mi cédula es 1.234.567-8 y mi tel es 099123456"

# Buscar un patrón de cédula uruguaya
cedula = re.search(r"\d.\d{3}.\d{3}-\d", texto)
if cedula:
    print(f"Cédula encontrada: {cedula.group()}")

# Salida: Cédula encontrada: 1.234.567-8
```



Los bloques básicos de regex

Símbolo	Significado	Ejemplo	Match
.	Cualquier carácter	a.c	"abc", "a1c", "a-c"
\d	Un dígito (0-9)	\d{3}	"123", "456"
\w	Letra, dígito o _	\w+	"hola", "var_1"
\s	Espacio en blanco	\s+	" ", "\t"
+	Uno o más	\d+	"1", "123", "99999"
*	Cero o más	\d*	"" , "5", "123"
?	Cero o uno	\d?	"" , "5"
{n}	Exactamente n	\d{3}	"123"
{n,m}	Entre n y m	\d{2,4}	"12", "1234"
[]	Conjunto	[aeiou]	"a", "e", "i"
^	Inicio de línea	^Hola	"Hola mundo"
\$	Fin de línea	fin\$	"es el fin"
	O (alternativa)	si no	"si", "no"
()	Grupo de captura	(\d{3})	captura "123"



Consejo: no hace falta memorizar todo. Se aprende usándolo.

Funciones principales del módulo re

```
import re
```

```
texto = "Montevideo 11200, Salto 50000, Paysandú 60000"
```

re.search() — primera coincidencia

```
re.search(r"\d{5}", texto)  
# Match: '11200'
```

re.findall() — todas las coincidencias

```
re.findall(r"\d{5}", texto)  
# ['11200', '50000', '60000']
```

re.sub() — busca y reemplaza

```
re.sub(r"\d{5}", "XXXXX", texto)  
# 'Montevideo XXXXX, Salto  
XXXXX, Paysandú XXXXX'
```

re.match() — solo al inicio del string

```
re.match(r"Montevideo", texto)  
# Match: 'Montevideo'
```

re.split() — divide por patrón

```
re.split(r",\s*", texto)  
# ['Montevideo 11200', 'Salto 50000', 'Paysandú  
60000']
```

Caso real 1: validar cédulas uruguayas

Formato de CI uruguaya: X.XXX.XXX-X (con o sin puntos y guión)

Validar formato

```
import re

def validar_ci(ci):
    """Valida formato de cédula uruguaya."""
    # Acepta: 1.234.567-8, 1234567-8, 12345678
    patron = r"^\d{1}?.?\d{3}?.?\d{3}-?\d{1}$"
    return bool(re.match(patron, ci.strip()))

# Tests
print(validar_ci("1.234.567-8")) # True
print(validar_ci("12345678"))   # True
print(validar_ci("123"))        # False
print(validar_ci("ab.cde.fgh-i")) # False
```

Normalizar formato

```
def normalizar_ci(ci):
    """Convierte cualquier formato
    de CI a X.XXX.XXX-X"""
    digitos = re.sub(r"^\d+", "", ci)
    # solo dígitos
    if len(digitos) != 8:
        return None
    return (
        f"{digitos[0]}."
        f"{digitos[1:4]}."
        f"{digitos[4:7]}-"
        f"{digitos[7]}"
    )
```

```
print(normalizar_ci("12345678")) # 1.234.567-8
print(normalizar_ci("1.234.567-8")) # 1.234.567-8
```

Caso real 2: extraer datos de texto libre

Situación: tienen un campo de texto con información mezclada.

```
comentarios = [  
    "Paciente de 45 años, peso 78.5kg, altura 1.72m",  
    "Edad: 32 años. Peso: 65 kg. Altura: 1.68 m",  
    "23 años, 90kg, 1.80m, fumador",  
]
```

Extraer edad, peso y altura

```
def extraer_datos_paciente(texto):  
    edad  = re.search(r"(\d{1,3})\s*años", texto)  
    peso  = re.search(r"(\d{2,3}(?:\.\d+)?)\s*kg", texto)  
    altura = re.search(r"(\d.\d{2})\s*m", texto)  
  
    return {  
        "edad": int(edad.group(1)) if edad else None,  
        "peso": float(peso.group(1)) if peso else None,  
        "altura": float(altura.group(1)) if altura else None,  
    }  
  
for c in comentarios:  
    print(extraer_datos_paciente(c))  
  
# {'edad': 45, 'peso': 78.5, 'altura': 1.72}  
# {'edad': 32, 'peso': 65.0, 'altura': 1.68}  
# {'edad': 23, 'peso': 90.0, 'altura': 1.80}
```

¿Dónde se usa esto?



Datos de salud



Encuestas abiertas



Logs de sistemas

Esto es muy común en datos de salud, encuestas abiertas y logs.

Caso real 3: limpiar datos sucios en el pipeline

Integración con el pipeline de la clase pasada:

```
def limpiar_telefonos(datos, campo="telefono"):
    """Normaliza teléfonos uruguayos
    al formato 09X XXX XXX."""
    patron = r"(?:\+598 | 0)?(\d{2})[\s-]?(\d{3})[\s-]?(\d{3})"

    for fila in datos:
        tel = fila.get(campo, "")
        match = re.search(patron, tel)
        if match:
            fila[campo] = (
                f"0{match.group(1)} "
                f"{match.group(2)} "
                f"{match.group(3)}"
            )
        else:
            fila[campo] = "INVALIDO"
    return datos
```

Transformaciones

→ +598 99 123 456 → **099 123 456**

→ 099-456-789 → **099 456 789**

→ tel: 098765432 → **098 765 432**

→ hola → **INVALIDO**

❏ Esto se agrega como una **etapa más del pipeline**.

Errores comunes con regex

Error 1: regex demasiado permisivo

```
# Malo: acepta cualquier cosa con dígitos
r"\d+"

# Mejor: ser específico sobre el formato esperado
r"0\d{2}\s?\d{3}\s?\d{3}"
```

Error 2: olvidar caracteres especiales

```
# El punto en regex significa "cualquier carácter"
# Para buscar un punto literal, hay que escaparlo

r"\d\.\d{3}" # correcto: busca "1.234"
r"\d.\d{3}"  # incorrecto: "1" + cualquier cosa + 3
             dígitos
```

Error 3: no usar raw strings

```
# Malo: Python interpreta \d como escape
"\d+"

# Bien: raw string, Python no interpreta escapes
r"\d+"
```

Error 4: regex ilegible

```
# Incomprensible:
r"^(?:\+598|0)?(\d{2})[\s-]?(\d{3})[\s-]?(\d{3})$"

# Mejor: usar re.VERBOSE para comentar
```

```
patron = re.compile(r"""
    ^(?:\+598|0)? # prefijo opcional (+598 o 0)
    (\d{2})      # código de operador (99, 98, etc.)
    [\s-]?      # separador opcional
    (\d{3})      # primeros 3 dígitos
    [\s-]?      # separador opcional
    (\d{3})$     # últimos 3 dígitos
""", re.VERBOSE)
```


Actividad práctica

Consigna: Se agregó en webasignatura un archivo CSV de un "censo ficticio" de una localidad del interior con las siguientes columnas problemáticas:

nombre

Algunos tienen títulos ("Dr.", "Ing."), otros están en MAYÚSCULAS.

ci

Formatos mezclados (con y sin puntos/guión).

email

Algunos son inválidos.

direccion

Algunas tienen código postal, otras no.

Tarea (en equipos) :

01

Escribir una función `normalizar_nombre(nombre)` que quite títulos y ponga formato Título.

02

Escribir una función `normalizar_ci(ci)` que lleve todo a formato `X.XXX.XXX-X`.

03

Escribir una función `validar_email(email)` que retorne `True/False`.

04

Escribir una función `extraer_codigo_postal(direccion)` que extraiga el CP si existe.

05

Integrar las 4 funciones en el pipeline de la semana pasada.



Tiempo: 40 minutos. Bonus: hacer un reporte de cuántos registros tenían problemas en cada campo.

Conexión con el Proyecto 1

¿Cómo usar regex en tu proyecto?

Revisen su dataset y pregúntense:



Texto libre

¿Hay campos de texto libre que necesiten extracción de datos?



Formatos inconsistentes

¿Hay campos con formatos inconsistentes que necesiten normalización?



Validación

¿Hay valores que necesiten validación contra un patrón?

Tu pipeline del Proyecto 1 ahora debería tener:

cargar → limpiar_faltantes → normalizar_con_regex → validar → transformar → exportar

Para la próxima clase:

- Integrar al menos una función con regex en el pipeline del proyecto.
- Documentar qué patrones encontraron y qué decisiones tomaron.

"Regex es una herramienta precisa para problemas de texto. Aprender a usarla bien les va a ahorrar horas de trabajo manual."